

# Lab5 Report

Jiaying Yong

## 1 Abstract

This report implements an end-to-end pipeline for hardware description language (HDL) code generation and verification: Gemini produces a structured prompt, DeepSeek generates Verilog code from that prompt, and Yosys performs synthesizability and basic property checks. Three representative tasks are used as targets—`binary_to_BCD` (combinational), `sequence_detector` (sequential), and `dice_roller` (LFSR-based). Because no large-scale experiments were conducted, the report does not present statistical accuracy. Instead, it provides a one-time engineering judgment: the pipeline is more stable on simple combinational logic; sequential tasks are mainly constrained by reset handling and state coverage; structured prompting clearly reduces common failure modes; and a minimal fix loop can raise the probability of first-pass success.

## 2 Introduction

Automatic HDL generation must satisfy both synthesizability and functional soundness. This work combines prompt engineering with formal tooling: a language-oriented model (Gemini) converts task descriptions into a constraint-explicit prompt; a reasoning-capable model (DeepSeek) emits Verilog; and Yosys serves as an automated judge for synthesis and basic checks. The goal is a low-overhead pipeline that still achieves a high first-pass success rate on common textbook-level tasks.

## 3 Experimental Setup

### 3.1 Pipeline

- Prompting (Gemini). Convert each task into a structured template containing: Interface (bit widths, reset polarity, sequential vs. combinational), Rules (coding style and synthesizable constraints), Verification Hints (boundary cases or timing cues), and Deliverables (module name and style expectations).
- Code Generation (DeepSeek). Emit Verilog-2001 following: `always_comb` with blocking assignments for combinational logic; `always_ff` with non-blocking assignments for sequential logic; FSMs written with `typedef enum`, explicit `next_state` separation, and a default path; no multi-driver nets and no combinational loops.
- Evaluation (Yosys). Run `read_verilog` → `synth` and categorize failures: width mismatches, inferred latches, multi-driven nets, missing/ambiguous reset, or combinational loops. To limit scope, no complex testbenches or formal proofs are included—only synthesis and basic property checks.

### 3.2 Task Coverage and Sampling

- `binary_to_BCD`: Double-Dabble, purely combinational.
- `sequence_detector`: Detect pattern “1101” with overlap; Mealy FSM.
- `dice_roller`: LFSR-driven values 1..6 with rejection sampling and valid signaling.

For each task, multiple candidates may be generated as needed (no `pass@k` is reported here), focusing on failure modes and first-pass outcomes.

## 4 Inconsistency Analysis and “Minimal Fix” Loop

### 4.1 Common Failure Types

- Implicit latches: Incomplete paths or missing defaults in combinational blocks.
- Width mismatches: Port/wire width inconsistencies causing truncation/extension.
- Blocking vs. non-blocking misuse: Race conditions from = inside sequential blocks.
- FSM/reset issues: Missing default, unreachable states, or ambiguous reset state.
- Multi-driven nets / combinational loops: Registers assigned in multiple always blocks or feedback without registers.

### 4.2 Minimal Fix Strategy

- Log-guided localization: Parse Yosys messages (e.g., latch inferred, width mismatch, multi-driven, combinational loop).
- Template patches:

Latch → add consistent default assignments in always\_comb.

Width mismatch → add explicit slicing/extension and verify declarations.

FSM → use typedef enum, include default, and reset explicitly to S0.

Multi-driven → ensure a register is assigned in one always\_ff only.

Comb loop → insert a register or redesign as sequential logic.

- Second pass generation: Append “failure reason + patch rule” to the prompt and regenerate with minimal edits; re-run Yosys.
- Prompt hardening: Feed effective patches back into the general prompt template to improve first-pass quality over time.

## 5 Representative Prompt Templates

### 5.1 binary\_to\_BCD

Task: Convert 8-bit unsigned binary to three BCD digits (hundreds, tens, ones).

I/O: input [7:0] bin; output logic [3:0] hundreds, tens, ones;

Rules:

- Pure combinational; one always\_comb; explicit widths; no latches.
- Use double-dabble or equivalent.

Checks: 0x00→000, 0x63→099, 0x64→100, 0xFF→255.

Deliverable: module binary\_to\_BCD (Verilog-2001).

### 5.2 sequence\_detector — detect “1101” with overlap

Task: Mealy FSM asserts `found` for one cycle when the stream contains "1101"; overlap allowed.

I/O: input clk, rst\_n, in\_bit; output logic found;

Rules:

- Active-low async reset to S0.
- typedef enum logic states; split always\_ff (state/output) and always\_comb (next\_state).
- Non-blocking in sequential; default path in combinational; no latches.

Checks: ...0011011... triggers once; "11011" triggers twice.

Deliverable: module sequence\_detector.

### 5.3 dice\_roller — 1..6 via LFSR and rejection sampling

Task: Dice roller outputs 1..6 using an LFSR PRNG with rejection sampling.

I/O: input clk, rst\_n, enable; output logic [2:0] value; output logic valid;

Rules:

- Update LFSR on enable; map 3'b001..110→1..6; reject others; assert valid on new legal sample; hold last value otherwise.
- Single clock; reset to a legal value; explicit widths; no multi-driven nets.

Deliverable: module dice\_roller with parameterizable LFSR taps.

## 6 Conclusion

The two-stage pipeline is most reliable on simple combinational designs (e.g., binary\_to\_BCD) under clear width and assignment rules.

For sequential tasks (e.g., sequence\_detector), first-pass success improves significantly when the prompt explicitly specifies reset polarity, pulse semantics, overlap behavior, and <=/= usage.

For LFSR-based designs (dice\_roller), correctness hinges on unambiguous rejection sampling and valid timing; otherwise, state holding and multi-driver errors are likely.

Encoding frequent failures and their fixes into the prompt template systematically improves initial code quality.

Even without complex testbenches, the pipeline provides a minimal yet submission-ready validation path; stronger guarantees can be added later via lightweight testbenches and formal assertions without altering the conclusions above.